

TRIN-FOR-TRIN

Onboarding-checklisten i Blazor — fra tom mappe til hostet app

Sådan laver du et nyt, rent Blazor WebAssembly-projekt der kun indeholder Checklist-varianten, kører det lokalt, og publicerer en hurtig Release-version du kan dele.

Hvert trin i denne guide er udført og verificeret på den kode der ligger i `proto/BlazorProto` (commit `fddb995`). Følger du dem i rækkefølge, ender du med det samme resultat — og det færdige projekt ligger som reference i `OnboardingChecklist/` i repoet.

new-design-project · september 2026 · .NET 8

0 Før du starter

- **.NET 8 SDK.** Tjek med `dotnet --version` — du skal se `8.x`. Guiden er testet på 8.0.130.
- **Kildekoden.** Vi kopierer filer fra den eksisterende prototype. I kommandoerne nedenfor står `$SRC` for stien til den:

```
SRC=~/.dev/emil-kowalski/new-design-project/proto/BlazorProto
```

- **En terminal.** Alt her er kommandolinje + en teksteditor. Ingen Visual Studio nødvendig.

Hvorfor et nyt projekt og ikke bare slette de to andre varianter? Fordi prototype-harneset (picker, tastaturnavigation, variant-skift, `?v=N` i URL'en) er filtret ind i `Proto.razor`, `Picker.razor`, `picker.js` og en tredjedel af CSS'en. Det er hurtigere og renere at starte fra nul og hente præcis de filer checklisten bruger.

1 Byg projektet

1 Opret et tomt Blazor WASM-projekt

```
dotnet new blazorwasm -e -o OnboardingChecklist
cd OnboardingChecklist
```

`-e` betyder *empty*: ingen Bootstrap, ingen eksempelsider. Templaten laver stadig en router, et layout og en Home-side, som vi ikke skal bruge — slet dem sammen med dens CSS og index.html:

```
rm -rf App.razor Pages Layout wwwroot/css wwwroot/index.html
mkdir -p Model Components wwwroot/css wwwroot/js
```

Appen er én side uden navigation, så en `Router` og et `MainLayout` er kun støj. Root-komponenten monteres direkte i `Program.cs` (trin 3).

2 Slå kulturdata fra i .csproj

Åbn `OnboardingChecklist.csproj` og tilføj to linjer i `<PropertyGroup>`, lige efter `<ImplicitUsings>`:

```
OnboardingChecklist.csproj
<PropertyGroup>
  <TargetFramework>net8.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
  <InvariantGlobalization>true</InvariantGlobalization>
  <BlazorEnableTimeZoneSupport>false</BlazorEnableTimeZoneSupport>
</PropertyGroup>
```

Appen formaterer ingen datoer eller tal efter kultur. Uden de to linjer sender Blazor ~2,5 MB ICU- og tidszonedata med i hvert load, som aldrig bliver brugt.

3 Erstat Program.cs

```
Program.cs

using Microsoft.AspNetCore.Components.WebAssembly.Hosting;
using OnboardingChecklist;
using OnboardingChecklist.Model;

var builder = WebAssemblyHostBuilder.CreateDefault(args);

// Single page, no router: Onboarding is the root component.
builder.RootComponents.Add<Onboarding>("#app");

// Singleton so answers survive re-renders.
builder.Services.AddSingleton<OnboardingState>();

await builder.Build().RunAsync();
```

4 Erstat _Imports.razor

```
_Imports.razor

@using Microsoft.AspNetCore.Components.Web
@using Microsoft.JSInterop
@using OnboardingChecklist
@using OnboardingChecklist.Model
@using OnboardingChecklist.Components
```

Filen gælder for alle `.razor`-filer i projektet, så komponenterne kan referere til `OnboardingState`, `Content`, `Icons` osv. uden egne `@using`-linjer.

5 Kopiér datamodellen

```
cp $SRC/Model/Content.cs $SRC/Model/OnboardingState.cs Model/
```

I begge filer: skift første linje fra `namespace BlazorProto.Model;` til `namespace OnboardingChecklist.Model;`.

I `Model/OnboardingState.cs` skal de felter der kun fandtes for pickeren og de to andre varianter væk. Find blokken *picker* + *per-variant navigation* og slet de markerede linjer:

Model/OnboardingState.cs

```
public int ActiveVariant { get; set; }
public int MountCounter { get; set; }

public int StepperIndex { get; set; }
public bool StepperDone { get; set; }

public int ConvoIndex { get; set; }
public bool ConvoDone { get; set; }

public string ChecklistKey { get; set; } = "name";
public bool ChecklistDone { get; set; }
```

Og nederst i `ResetFlow()`:

```
public void ResetFlow()
{
    Errors.Clear();
    StepperIndex = 0; StepperDone = false;
    ConvoIndex = 0; ConvoDone = false;
    ChecklistKey = "name"; ChecklistDone = false;
}
```

`Content.cs` er alt indholdet — de fire opgaver, rollelisten, notifikationsvalgene, workspace-navnet. `OnboardingState.cs` er svarene, valideringen og de afledte værdier (hvor mange er færdige, hvad mangler). Begge er ren C# uden Blazor-afhængigheder, så de flytter med uændret.

6 Kopiér de delte komponenter

```
cp $SRC/Components/Icons.cs $SRC/Components/Recap.razor $SRC/Components/Fields.razor
Components/
```

- `Icons.cs`: skift namespace til `OnboardingChecklist.Components`.
- `Recap.razor`: ingen ændringer.
- `Fields.razor`: én ændring. Feltfokus går gennem et lille JS-kald, og funktionen skifter navn fra picker-objektet til vores eget:

Components/Fields.razor

```
public ValueTask FocusFirstAsync() => JS.InvokeVoidAsync("protoPicker.focusFirst", root);
public ValueTask FocusFirstAsync() => JS.InvokeVoidAsync("app.focusFirst", root);
```

`Fields` renderer felterne for én opgave ad gangen (navn, rolle, invitationer, notifikationer) og var delt mellem Stepper og Checklist. Kommentaren øverst i filen om Stepper kan du slette — den passer ikke længere.

7 Kopiér checklisten som root-komponent

```
cp $SRC/Variants/Checklist.razor Onboarding.razor
```

Ingen ændringer i indholdet. Filnavnet bliver klassenavnet, så `Onboarding.razor` giver komponenten `Onboarding` — den som `Program.cs` monterer i `#app`.

8 Lav `wwwroot`

JavaScript. Én funktion, otte linjer — resten af `picker.js` var harness:

```
wwwroot/js/app.js

// Blazor's FocusAsync() needs an ElementReference per field; one helper that
// finds the first input in a container is far less ceremony.
window.app = {
  focusFirst(container) {
    if (!container) return;
    const el = container.querySelector('input:not([type=hidden]), textarea');
    (el || container).focus();
  },
};
```

index.html. Skriv filen sådan (boot-baren udfyldes af Blazor via `--blazor-load-percentage`, og fejl-elementet er skjult indtil der sker en rigtig fejl — begge dele styles i CSS'en):

```
wwwroot/index.html

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Onboarding</title>
  <base href="/" />
  <link rel="stylesheet" href="css/app.css" />
  <link rel="icon" type="image/png" href="favicon.png" />
</head>
<body>
  <div id="app">
    <div class="boot" role="status" aria-label="Loading">
      <div class="boot-bar"><span class="boot-fill"></span></div>
    </div>

    <div id="blazor-error-ui">
      An unhandled error has occurred.
      <a href="" class="reload">Reload</a>
      <a class="dismiss"></a>
    </div>

    <script src="js/app.js"></script>
    <script src="_framework/blazor.webassembly.js"></script>
  </body>
</html>
```

Kopier også `favicon.png` fra `$SRC/wwwroot/`.

CSS. `$SRC/wwwroot/css/app.css` er delt op i fem sektioner med tydelige bannere (`/* ==== NAVN ====` `*/`). Lav en ny `wwwroot/css/app.css` med disse tre dele, i denne rækkefølge:

TAG MED	FRA ... TIL	HVORFOR
MATERIALS	Banneret <code>MATERIALS</code> til og med <code>@media</code> (<code>forced-colors: active</code>) { ... }	Tokens, base, knapper, felter, boot-bar, fejl-bjælke
Tick + recap	Fra <code>.v1-tick</code> { til og med <code>.recap span</code> { ... } (inkl. <code>@keyframes pop</code>)	Bruges på "You're all set"- skærmen
VARIANT 3 — CHECKLIST	Banneret til filens slutning	Selve checklisten

Udelad: `PICKER` (øverst), resten af `VARIANT 1 — STEPPER` og hele `VARIANT 2 — CONVERSATIONAL` .
Resultatet er ca. 530 linjer mod 934.

Klassenavnene `v3-*` og `v1-tick` stammer fra prototypen. De virker som de er; omdøb dem først når appen kører.

9 Kør den

dotnet run

Terminalen skriver `Now listening on: http://localhost:XXXX` — åbn den adresse. Du skal se:

- ☐ En tynd lilla bar i midten under load, som fylder op (ikke tekst)
- ☐ "Finish setting up" med rail til venstre (2 of 4 complete, 50 %) og panelet "What's your name?" til højre
- ☐ Klik **Save and continue** → panelet skifter til "What do you do?"
- ☐ Vælg en rolle, gem, klik **Finish setup** → "You're all set, Bart." med recap
- ☐ Ingen tekst nederst på siden, ingen scrollbar

Får du gammel visning efter en ændring: hard reload (`Ctrl+Shift+R`). Blazor cacher runtime-filerne aggressivt.

10 Hvis noget fejler

SYMPTOM	ÅRSAG	LØSNING
The type or namespace name 'BlazorProto' could not be found	Et namespace blev ikke skiftet	Søg efter <code>BlazorProto</code> i alle filer; skift til <code>OnboardingChecklist</code>
'OnboardingState' does not contain a definition for 'StepperIndex'	Du slettede et felt der stadig bruges	Du har kopieret en anden variant end <code>Checklist.razor</code>
Fejl i konsollen: Could not find 'protoPicker.focusFirst'	Trin 6, JS-navnet	Ret til <code>app.focusFirst</code> i <code>Fields.razor</code>
Appen loader men er ustylet	<code>index.html</code> peger på forkert CSS, eller CSS'en mangler <code>MATERIALS</code>	Tjek <code><link href="css/app.css"></code> og at <code>:root {</code> findes i filen
"An unhandled error has occurred" nederst, men appen virker	CSS-reglen for <code>#blazor-error-ui</code> mangler	Den ligger i <code>MATERIALS</code> — sektion ikke kopieret helt

2 Publish og host

`dotnet run` er et Debug-build: 193 separate `.wasm`-filer, 8,5 MB, ingen trimming. Det er fint til udvikling og forkert til alt andet. En Release-publish trimmer klassebiblioteket ned til det der faktisk bruges og komprimerer det.

1 Publicér

```
dotnet publish -c Release -o publish
```

Tager 10–20 sekunder. Det der skal på nettet ligger i `publish/wwwroot/` — alt andet i `publish/` kan ignoreres.

	FILER	REQUESTS	OVERFØRT	LOAD (LOCALHOST)
Debug (<code>dotnet run</code>)	193 <code>wasm</code>	202	8,5 MB	1,1 s
Release publish, brotli	30 <code>wasm</code>	38	1,8 MB	0,5 s

Målt på prototypen med de to `csproj`-flag fra trin 2. Ved 4 Mbit tager Debug-versionen 19 s og publish-versionen ca. 4 s — forskellen er størst for dem der ikke sidder på din maskine.

Ved siden af hver `.wasm` ligger en `.br` (brotli) og en `.gz`. En server der kender dem, sender den komprimerede udgave og sparer 70 %. En server der ikke gør, sender `.wasm` ukomprimeret (ca. 5,5 MB) — det virker stadig.

2 Test det lokalt

```
python3 -m http.server 8080 -d publish/wwwroot
```

Åbn `http://localhost:8080`. Python-serveren sender ukomprimeret, men det er 38 requests i stedet for 202, så det er stadig markant hurtigere end `dotnet run`. Stop med `Ctrl+C`.

3 Læg det online

Netlify Drop — hurtigst når du bare vil vise det til nogen:

1. Gå til `app.netlify.com/drop`
2. Træk mappen `publish/wwwroot` ind i browservinduet
3. Du får en URL med det samme. Netlify komprimerer selv, så du får brotli-hastigheden uden opsætning.

GitHub Pages — hvis det skal ligge ved siden af koden:

Bemærk: GitHub Pages er kun tilgængeligt for private repos med GitHub Pro. Med en gratis konto skal repoet være offentligt.

1. Lav en tom fil `publish/wwwroot/.nojekyll`. Uden den ignorerer Pages mappen `_framework/`, fordi den starter med underscore.
2. Sitet bliver serveret fra `https://<bruger>.github.io/<repo>/`, så ret `<base href="/" />` i `publish/wwwroot/index.html` til `<base href="/<repo>/" />`.
3. Push indholdet af `publish/wwwroot` til en `gh-pages` -branch (eller en `docs/` -mappe på main), og peg repoets *Settings* → *Pages* på den.

GitHub Pages sender ikke `.br` -filerne selv, så load er ~5,5 MB ukomprimeret. Stadig 38 requests og trimmet — helt brugbart.

4 Valgfrit: mindre runtime

```
sudo dotnet workload install wasm-tools
```

Med denne workload relinker publish selve .NET-runtimen (`dotnet.native.wasm`, den største enkelte fil på 2,8 MB) og fjerner det appen ikke bruger. Du behøver ikke ændre noget i projektet — næste `dotnet publish` bruger den automatisk. Kræver sudo, fordi SDK'en ligger i `/usr/lib/dotnet`.

A Appendiks: filerne i det færdige projekt

```
OnboardingChecklist/
├─ OnboardingChecklist.csproj    ← trin 2
├─ Program.cs                   ← trin 3
├─ _Imports.razor               ← trin 4
├─ Onboarding.razor             ← trin 7
(= Checklist.razor)
├─ Model/
|   ├─ Content.cs               ← trin 5
|   └─ OnboardingState.cs       ← trin 5,
trimmet
├─ Components/
|   ├─ Fields.razor             ← trin 6,
én linje rettet
|   └─ Icons.cs                 ← trin 6
|       └─ Recap.razor          ← trin 6
├─ Properties/
|   └─ launchSettings.json      (fra
templaten)
└─ wwwroot/
    ├─ index.html               ← trin 8
    ├─ favicon.png
    ├─ css/app.css              ← trin 8,
tre sektioner
    └─ js/app.js                ← trin 8
```

Filer fra prototypen der *ikke* skal med: `Proto.razor` , `Components/Picker.razor` , `Variants/Stepper.razor` , `Variants/Conversational.razor` , `wwwroot/js/picker.js` .